

generate_markdown_exports

scripts/ generate_markdown_exports.sh — the §11.4.65 document-twin exporter

Revision: 5 **Last modified:** 2026-09-26T14:26:37Z **Status:** active

Overview

scripts/generate_markdown_exports.sh is the project's canonical §11.4.65 exporter. For every in-scope *.md source it produces the sibling document twins:

Twin	Produced by	Present when
.html	pandoc --standalone, else python-markdown	always (one leg or the other must exist)
.pdf	weasyprint rendering the .html	the weasyprint CLI is on PATH
.docx	pandoc -f markdown -t docx (direct from the .md, BOB-011)	pandoc is on PATH

It is the script the pre-build sync invariant, scripts/compute-badges.sh, and scripts/testing/update_readme_doc_links.sh all sit downstream of.

Prerequisites

Every leg is **optional and independently degradable**, and a leg that cannot run writes **nothing** — the script never emits a blank or placeholder file (§11.4.3 honest SKIP, §11.4.6 no fabricated output).

- `pandoc` — HTML + DOCX. The highest-fidelity path.
- `weasyprint` (the **CLI**, not the Python module) — PDF.
- a Python interpreter that can import `markdown` — the HTML fallback used only when `pandoc` is absent.
- If neither `pandoc` nor a markdown-capable interpreter is found, the script **exits 1** and names both the interpreters it probed and the remedy. It does not half-run.

All three install without root:

```
uv tool install weasyprint #
    weasyprint CLI
uv pip install --python .venv/bin/python markdown # python-
    markdown fallback
# pandoc: a user-local tarball unpacked under ~/.local/opt +
    symlinked into ~/.local/bin
```

Usage

```
# Full in-scope sweep: project root *.md + docs/**/*.md + scripts/
    **.md
bash scripts/generate_markdown_exports.sh

# Explicit scope – regenerate ONLY these sources' twins
bash scripts/generate_markdown_exports.sh CLAUDE.md docs/
    USER_MANUAL.md
```

Inputs

Input	Meaning
<code>\$@</code> (optional)	Explicit <code>.md</code> paths. When present, only those files are converted and the discovery sweep is skipped entirely.
<code>\$BOBA_EXPORT_PYTHON</code> (optional)	Interpreter to use for the python-markdown leg. Absent → auto-resolved.

Outputs

.html, .pdf, and .docx siblings written next to each source .md. Per twin type the script prints how many were rendered, how many of those were byte-identical to the existing twin (and therefore not rewritten), and how many legs failed.

Every leg renders into a private scratch directory first and only then installs the result. The install never moves the render across filesystems (the scratch directory can be tmpfs, where mv degrades to unlink + copy): the bytes are copied to a temp file named `.<twin>.exporttmp.XXXXXX.tmp` in the twin's **own** directory, given the existing twin's mode (or the umask default for a new twin), and renamed over the twin with `mv -f` — an atomic `rename(2)`. A failed install removes the temp, leaves the previous twin untouched, prints an `ERROR: could not install ...` line and counts as a failed leg (never as “byte-identical”). A temp left behind by a `SIGKILL` matches the project's `*.tmp` ignore rule and no twin pattern.

Exit codes

Code	Meaning
0	Completed (including “nothing was stale”).
1	No HTML converter available at all, or an explicitly-named argument was not an existing .md file.
2	The run completed, but at least one HTML leg failed. Each failure is printed on stderr as an <code>ERROR: line</code> carrying the converter's own message.

A failed **PDF** or **DOCX** leg keeps its historical non-fatal status: it is counted, reported as a `WARN: line` on stderr, and the previous twin (if any) is left in place.

A failed HTML leg (BOB-249 review M4)

The HTML converter used to run bare under `set -e` with its stderr thrown away. One failing render therefore aborted the whole run with no message at all (measured: exit 3, the DOCX leg of the same file and every later file never processed). Now the failure is caught and reported, the stale .html is left exactly as it was, the .pdf is **not** re-

derived from that stale `.html` (a PDF baked from it would look fresh while carrying old content), the DOCX leg and every remaining file still run, and the script exits 2.

Why the explicit-scope argument exists

The full sweep touches ~355 exports. A session that edits nine documents should resync exactly those nine twins: re-rendering 346 unrelated exports costs minutes of `weasyprint` time and needlessly churns the corpus that `scripts/pre_build/check_cm_export_charset_valid.sh` measures against its monotone ratchet baseline. Passing the changed files keeps the blast radius equal to the change.

An argument that is not an existing `.md` file is a **hard error**, never a silent skip — a typo'd path must not be able to masquerade as “nothing to do” (§11.4.201(6): a quiet zero from a blind instrument is indistinguishable from a clean corpus).

Idempotency and the staleness rule

Re-running is a no-op: a second run over unchanged sources rewrites nothing and leaves every twin byte-identical.

Staleness uses the same oracle as the pre-build gate (BOB-249)

The writer decides whether a twin is stale with `scripts/lib/export_staleness.sh` — the **same** content-history oracle CM-MARKDOWN-EXPORT-SYNC (pre-build invariant 16) uses, so writer and gate agree on what “stale” means:

Situation	Verdict
twin missing	stale
<code>.md</code> (or twin) locally edited / untracked	stale when the twin's mtime is older (mtime is meaningful here)

Situation	Verdict
both committed and clean	stale only when the .md's last-touching commit is more recent than the twin's (git history, never mtime)
file outside any git work tree, or the lib missing next to the script	plain mtime fallback, announced on stderr when the lib is missing

Why not mtime: a fresh `git checkout` writes `x.docx` and `x.html` **before** `x.md` (path order), so on a scratch clone 226 of 456 .md files were strictly newer than their .docx and 42 than their .html. The old mtime-only writer rewrote all 310 of those twins with zero content change. With the oracle the same fresh checkout rewrites 0, while a real content edit of a .md still regenerates all three twins.

Byte-stable output (SOURCE_DATE_EPOCH)

pandoc stamps the wall-clock time into every .docx (docProps/core.xml), so each regeneration used to be a byte-different file. The script now exports `SOURCE_DATE_EPOCH` per file before rendering:

- the .md's last-commit time (`git log -1 --format=%ct -- <md>`), so the stamp is a property of the source history, identical in every FULL-history clone (a shallow clone stamps the boundary commit time);
- 1785674948 (2026-08-02T12:49:08Z, the value `constitution/scripts/render/render-governance-twins.sh` pins) for a never-committed .md;
- a `SOURCE_DATE_EPOCH` the caller already exported wins.

Regenerating identical content twice now yields a byte-identical `.docx`. weasyprint 69.0 PDFs were already byte-stable (measured); HTML from pandoc `--standalone` carries no timestamp. **Limit:** a twin rendered while its `.md` is still uncommitted carries the *previous* commit's time; a forced re-render after that `.md` is committed carries the new one. Output is deterministic per (content, history), not per content alone.

Identical renders are not rewritten, and HTML records its provenance (BOB-249 review M1)

Two rules together make a stale pair **converge** instead of staying stale forever:

1. **A render byte-identical to the existing twin is not rewritten.**
The twin is only re-stamped (touch), which records “verified against the current source” for the mtime half of the oracle, so a locally-edited or untracked source is not re-rendered on the next run.
2. **Every generated `.html` records what it was built from**, in two `<meta>` elements inserted just before the first `</head>` after pandoc has run (not via `pandoc -H`, which suppresses a document's own YAML header-`includes`; the bytes otherwise equal the former `-H` output): `x-export-source-sha256` (sha256 of the source bytes) and `dcterm.modified` (the per-file `SOURCE_DATE_EPOCH`, i.e. the source revision time). weasyprint maps `dcterm.modified` into the PDF's `ModDate`, so the `.pdf` moves with it.

Why rule 2 is needed: the gate judges a committed pair by git history. A whitespace-only `.md` commit made the `.md` newer than its twins, but pandoc and weasyprint rendered byte-**identical** `.html/.pdf` for it, so nothing could be committed and the gate called the pair stale forever (measured: only the `.docx`, whose stamp moved, showed up in `git status`). With the provenance record, a regeneration for a newer source revision always produces committable bytes; after committing them the gate calls the pair fresh. Both values are pure functions of (content, history) — never the wall clock — so output stays byte-stable. The oracle and the gate are unchanged.

Limit: when the caller pins one `SOURCE_DATE_EPOCH` for every revision and a later commit restores a `.md` to content an older twin was already built from, the render is identical again and the pair cannot converge by writing alone. Closing that residual case needs the staleness oracle itself to compare content fingerprints; that is an oracle/gate change and is **not** part of this script. Two `.md` commits within the same second share an epoch (only the `.html` sha then moves; the `.pdf` may not).

Docs Chain-owned twins are rendered exactly as the engine renders them (BOB-249 review I1)

A twin that is a node path: of any `.docs_chain/contexts/*.yaml` context (for example `docs/features/Status.html`) is owned by the Docs Chain engine, whose `verify` (pre-build invariant 24, CM-DOCS-CHAIN-ENGINE-VERIFY) recomputes it and compares **bytes**. For such a twin the script uses the engine's own `argv` from `constitution/submodules/docs_chain/internal/adapter/derived.go` — `pandoc --standalone --from=markdown --to=html --metadata title=<base>`, `pandoc --from=markdown --to=docx --metadata title=<base>`, `weasyprint --base-url <live pdf path>` — with `SOURCE_DATE_EPOCH=946684800` and **no** provenance tags, so running this script before `docs_chain sync` cannot make invariant 24 fail. Every other twin keeps the provenance tags. A context file with no parsable node path prints one `NOTE:` line and its files are treated as not owned. **Limit:** an engine-owned `.html` has no provenance record, so the rule-2 convergence above does not apply to it. Guard: `tests/unit/test_generate_markdown_exports_engine_parity.sh`.

Structural validity check (BOB-249 review M3)

History says nothing about **content**: a twin corrupted after it was generated (garbage bytes with a newer `mtime`, or a corruption that was committed) used to be judged “fresh” by writer and gate alike, forever. Each existing twin is now also checked for cheap **structural** validity, and an invalid one is regenerated:

Twin	Valid when
<code>.html</code>	it has a <code><meta ... charset></code> element and a closing <code></html></code>
<code>.pdf</code>	it starts with <code>%PDF-</code> and its last 1 KiB contains <code>%%EOF</code>
<code>.docx</code>	it starts with the zip local-header magic <code>PK\x03\x04</code> and (when <code>unzip</code> is on <code>PATH</code>) lists <code>word/document.xml</code>

This is deliberately **not** a byte comparison with a fresh render — that would re-render the whole corpus on every run. A fresh checkout still rewrites **0** twins (measured over all 464 in-scope sources). The checks

never pipe a producer into `grep -q`: an early `grep` exit kills the producer with `SIGPIPE`, and under `pipefail` a valid twin would read as invalid and be regenerated on every run (§11.4.201(12)).

The charset self-heal rule

Staleness is **not** history alone. An `.html` that exists and is `mtime-fresh` but declares **no charset** is treated as `STALE` and regenerated, because such a file is a pre-BOB-169 charset-less fragment that would otherwise persist forever and keep re-baking mojibake into any PDF rendered from it. PDF staleness additionally keys on the `.html` (the PDF is derived from it), so healing the HTML half cannot leave a corrupt PDF behind. An HTML regenerated in the **current** run always re-derives its PDF explicitly: the oracle caches the working-tree state once per run, so to it a just-rewritten HTML still looks clean.

Internal behaviour worth knowing

Capability probes test the artifact, not a prerequisite

The `weasyprint` probe runs `weasyprint --version` — the real entry point the render path uses. It previously also required `python3 -c "from weasyprint import HTML"` to succeed, which is a **prerequisite** probe, not an artifact probe: a `weasyprint` CLI installed into its own isolated environment (`uv tool install`, `pipx`, a `venv` on `PATH`) renders PDFs perfectly while being invisible to a system-interpreter import. On this host that made the script announce *“PDF support: not available”* and produce **zero PDFs while exiting 0** — a false negative of exactly the shape §11.4.201(11) names. Do not reintroduce an import-based probe for a CLI-invoked tool.

The python-markdown interpreter is resolved by trying the import

`PY_MD` is chosen by actually attempting `import markdown` against, in order: `$BOBA_EXPORT_PYTHON`, the repo’s `.venv/bin/python`, then `python3`. It is never assumed from a path’s existence (§11.4.6). Probing only system `python3` is insufficient here: this host’s `/usr/bin/python3` has no `pip` and is `PEP-668 EXTERNALLY-MANAGED`, so `markdown` cannot live there without `root` — while the repo `.venv` holds it with no privilege at all.

Encoding is pinned on both legs

The corpus carries Cyrillic (Боба) and §. The python-markdown leg opens and writes with an explicit `encoding='utf-8'` rather than inheriting the ambient locale, so a C/POSIX locale cannot mangle the read and leave the emitted `<meta charset="utf-8">` advertising a lie.

Verifying the output

```
# charset declared on every generated export (ratcheted gate)
bash scripts/pre_build/check_cm_export_charset_valid.sh

# the PDF text layer really preserves non-ASCII
bash tests/unit/test_export_pdf_charset_integrity.sh

# DOCX shape + idempotency
bash tests/unit/test_docx_export.sh

# checkout-order touch rewrites nothing; content edit / missing twin /
# charset fragment still regenerate; docx + pdf byte-stable
bash tests/unit/
    test_generate_markdown_exports_content_staleness.sh

# the shared staleness oracle (incl. .docx history)
bash tests/unit/test_export_staleness_oracle.sh

# M1 convergence after a whitespace-only commit, M3 corrupt-twin healing with a
# valid-twin control, M4 loud non-fatal HTML failure, plus the (a) content
# change / (b) missing twin / (c) charset fragment regeneration controls
bash tests/unit/test_generate_markdown_exports_review_followups.sh

# I1 docs_chain engine byte parity (real engine verify when built), M-a YAML
# header-includes kept, M-b atomic same-directory install keeping the mode
bash tests/unit/test_generate_markdown_exports_engine_parity.sh
```

When checking a heading survived into the HTML, match against a **whitespace-normalized** stream, not lines: pandoc hard-wraps its HTML at ~72 columns, so a heading routinely spans a newline and a line-oriented `grep -F` returns a confident, wrong zero (§11.4.201(7)(c)). Remember also that `&` is correctly emitted as `&`;

Related scripts

- `scripts/pre_build/check_cm_export_charset_valid.sh` — the gate over this script's output; reads a monotone-decreasing ratchet baseline.
- `scripts/regenerate-continuation-exports.sh` — deliberately narrow CONTINUATION-only regen; see its own guide for why it does not call this.
- `scripts/lib/export_staleness.sh` — the staleness oracle shared by this script and the pre-build gate.
- `tests/unit/test_export_pdf_charset_integrity.sh` — the paired §1.1 guard.

Last verified: 2026-09-26 — fresh scratch-clone checkout of `docs/ + README.*` (456 `.md`): 310 twins rewritten before the fix, 0 after; a control edit still regenerated its three twins. Content-staleness, oracle, DOCX, PDF charset-integrity and path-argument guards all PASS. Re-verified after the review follow-ups (M1/M3/M4): explicit-scope run over all 464 in-scope sources of a fresh clone rewrote 0 twins (content and mtime); a corrupted `.docx` planted as a needle was the only one regenerated; the follow-up suite passes 18/18 (11 of its assertions failed before).